Com S // Cpr E // Math 5250

Numerical Analysis of High-Performance Computing

Instructor: Songting Luo

Lecture 17: Amdahl's & Gustafson's Law,
Introduction to OpenMP,
and Using the ISU HPC-Class Cluster

Outline

1. Amdahl's & Gustafson's Law
2. Introduction to OpenMP
3. Using the ISU HPC-Class Cluster

Amdahl’s & Gustafson’s Law

Amdahl's Law

Typically only part of a computation can be parallelized.

Question:

Suppose 50% of the computation is inherently sequential, and the other 50% can be parallelized.

How much faster could the computation potentially run on many processors?

Amdahl's Law

Typically only part of a computation can be parallelized.

Question:

Suppose 50% of the computation is inherently sequential, and the other 50% can be parallelized.

How much faster could the computation potentially run on many processors?

Answer:

At most a factor of 2, no matter how many processors.

The sequential part is taking $\frac{1}{2}$ the time and that time is still required even if the parallel part is reduced to zero time.

Amdahl's Law

Typically only part of a computational can be parallelized.

Question:

Suppose 10% of the computation is inherently sequential, and the other 90% can be parallelized.

How much faster could the computation potentially run on many processors?

Amdahl's Law

Typically only part of a computational can be parallelized.

Question:

Suppose 10% of the computation is inherently sequential, and the other 90% can be parallelized.

How much faster could the computation potentially run on many processors?

Answer:

At most a factor of 10, no matter how many processors.

The sequential part is taking $\frac{1}{10}$ the time and that time is still required even if the parallel part is reduced to zero time.

Amdahl's Law

Suppose $\frac{1}{\alpha}$ of the computation is inherently sequential, and the other $\left(1 - \frac{1}{\alpha}\right)$ can be parallelized.

Then can gain at most a factor of α, no matter how many processors.

Amdahl's Law

Suppose $\frac{1}{\alpha}$ of the computation is inherently sequential, and the other $\left(1 - \frac{1}{\alpha}\right)$ can be parallelized.

Then can gain at most a factor of α, no matter how many processors.

If T_S is the time required on a sequential machine and we run on P processors, then the time required will be (at least):

$$T_P = \left(\frac{1}{\alpha}\right) T_S + \left(1 - \frac{1}{\alpha}\right) \frac{T_S}{P} = \left(\frac{P + \alpha - 1}{P\alpha}\right) T_S$$

Amdahl's Law

Suppose $\frac{1}{\alpha}$ of the computation is inherently sequential, and the other $\left(1 - \frac{1}{\alpha}\right)$ can be parallelized.

Then can gain at most a factor of α, no matter how many processors.

If T_S is the time required on a sequential machine and we run on P processors, then the time required will be (at least):

$$T_P = \left(\frac{1}{\alpha}\right) T_S + \left(1 - \frac{1}{\alpha}\right) \frac{T_S}{P} = \left(\frac{P + \alpha - 1}{P\alpha}\right) T_S$$

$$T_P \to \left(\frac{1}{\alpha}\right) T_S \qquad \text{as} \qquad P \to \infty$$

Amdahl's Law

Suppose $\frac{1}{\alpha}$ of the computation is inherently sequential, and the other $\left(1 - \frac{1}{\alpha}\right)$ can be parallelized.

Then can gain at most a factor of α, no matter how many processors.

If T_S is the time required on a sequential machine and we run on P processors, then the time required will be (at least):

$$T_P = \left(\frac{1}{\alpha}\right) T_S + \left(1 - \frac{1}{\alpha}\right) \frac{T_S}{P} = \left(\frac{P + \alpha - 1}{P\alpha}\right) T_S$$

$$T_P \to \left(\frac{1}{\alpha}\right) T_S \qquad \text{as} \qquad P \to \infty$$

Therefore, the **speedup** is

$$\frac{T_S}{T_P} = \frac{P\alpha}{P + \alpha - 1} = \frac{\alpha}{1 + \frac{\alpha - 1}{P}} \to \alpha \qquad \text{as} \qquad P \to \infty$$

Gustafson's Law

The problem with Amdahl's Law is that it considers only a fixed problem size.

In practice, if we increase the number of processors, we also increase the problem size (roughly the amount of work that a single processor does remains constant with changing P).

Gustafson's Law

The problem with Amdahl's Law is that it considers only a fixed problem size.

In practice, if we increase the number of processors, we also increase the problem size (roughly the amount of work that a single processor does remains constant with changing P).

Gustafson's Law: In the multi-processor case, time spent on 1 processor is

$$\text{(sequential part time) + (parallel time)} : \quad a + b$$

If we tried to do this on a single processor (doing the same total amount of work) it would take

$$\text{(sequential part time) + } P\times \text{ (parallel time)} : \quad a + Pb$$

We define the ratio:

$$\alpha = \frac{a}{a+b} \qquad \Longrightarrow \qquad 1 - \alpha = \frac{b}{a+b}$$

Gustafson's Law

The problem with Amdahl's Law is that it considers only a fixed problem size.

In practice, if we increase the number of processors, we also increase the problem size (roughly the amount of work that a single processor does remains constant with changing P).

Gustafson's Law: In the multi-processor case, time spent on 1 processor is

$$\text{(sequential part time) + (parallel time)} : \quad a + b$$

If we tried to do this on a single processor (doing the same total amount of work) it would take

$$\text{(sequential part time) + } P\times \text{ (parallel time)} : \quad a + Pb$$

We define the ratio:

$$\alpha = \frac{a}{a+b} \qquad \Longrightarrow \qquad 1 - \alpha = \frac{b}{a+b}$$

Now define the **speedup**:

$$S = \frac{a+Pb}{a+b} = \alpha + P\left(1-\alpha\right).$$

Furthermore, if $\alpha \to 0$ as $P \to \infty$, then we get the optimal scaling that $S \to P$.

Amdahl's Law vs. Gustafson's Law: an analogy

Amdahl's law approximately suggests:

> *Suppose a car is traveling between two cities 60 miles apart, and has already spent one hour traveling half the distance at 30 mph. No matter how fast you drive the last half, it is impossible to achieve 90 mph average before reaching the second city. Since it has already taken you 1 hour and you only have a distance of 60 miles total; going infinitely fast you would only achieve 60 mph.*

Gustafson's law approximately states:

> *Suppose a car has already been traveling for some time at less than 90mph. Given enough time and distance to travel, the car's average speed can always eventually reach 90mph, no matter how long or how slowly it has already traveled. For example, if the car spent one hour at 30 mph, it could achieve this by driving at 120 mph for two additional hours, or at 150 mph for an hour, and so on.*

Source: `https://en.wikipedia.org/wiki/Gustafson's_law`

Scaling

Some algorithms scale better than others as the number of processors increases.

Typically interested in how well algorithms work for large problems:

- Particle methods with N particles
- Algorithms for solving systems of N algebraic or differential equations
- Algorithms for solving PDEs on $N \times N \times N$ grid in 3D

Scaling

Some algorithms scale better than others as the number of processors increases.

Typically interested in how well algorithms work for large problems:

- Particle methods with N particles
- Algorithms for solving systems of N algebraic or differential equations
- Algorithms for solving PDEs on $N \times N \times N$ grid in 3D

For large N, there may be lots of inherent parallelism.

Scaling

Some algorithms scale better than others as the number of processors increases.

Typically interested in how well algorithms work for large problems:

- Particle methods with N particles
- Algorithms for solving systems of N algebraic or differential equations
- Algorithms for solving PDEs on $N \times N \times N$ grid in 3D

For large N, there may be lots of inherent parallelism.

But this depends on many factors:

- Dependencies between calculations
- Communication as well as flops
- Nature of problem and algorithm chosen

Scaling

Typically interested on how well algorithms work for large problems.

Strong scaling: How does the algorithm perform as the number of processors P increases for a fixed problem size N?

Any algorithm will eventually break down (for example when $P > N$ or Amdahl's law)

Weak scaling: How does the algorithm perform when the problem size increases with the number of processors?

e.g., if we double the number of processors can we solve a problem "twice as large" in the same CPU time?

Weak scaling

What does “twice as large” mean?

Depends on how algorithm complexity scales with N.

Example: sovling an $N \times N$ linear system with Gaussian elimination requires $\mathcal{O}(N^3)$ flops.

Doubling N requires 8 times as many operations

Weak scaling

What does “twice as large” mean?

Depends on how algorithm complexity scales with N.

Example: sovling an $N \times N$ linear system with Gaussian elimination requires $\mathcal{O}(N^3)$ flops.

Doubling N requires 8 times as many operations

Problem is “twice as large” if we increase N by a factor of $2^{1/3} \approx 1.26$, e.g., from 100×100 to 126×126.

(Or may be better to count memory accesses!)

Weak scaling

Solving steady state heat equation with $N \times N \times N$ grid.

N^3 grid points $\Longrightarrow$ linear system with this many unknowns.

If we used Gaussian elimination (very bad idea!) we would require $\sim (N^3)^3 = N^9$ flops.

Double N would require $2^9 = 512$ times more flops.

Weak scaling

Solving steady state heat equation with $N \times N \times N$ grid.

N^3 grid points $\Longrightarrow$ linear system with this many unknowns.

If we used Gaussian elimination (very bad idea!) we would require $\sim (N^3)^3 = N^9$ flops.

Double N would require $2^9 = 512$ times more flops.

Good iterative methods can do the job in $\mathcal{O}(N^3)\ \log_2(N)$ flops or less (e.g., multigrid).

Weak scaling

Solving steady state heat equation with $N \times N \times N$ grid.

N^3 grid points $\Longrightarrow$ linear system with this many unknowns.

If we used Gaussian elimination (very bad idea!) we would require $\sim (N^3)^3 = N^9$ flops.

Double N would require $2^9 = 512$ times more flops.

Good iterative methods can do the job in $\mathcal{O}(N^3)\ \log_2(N)$ flops or less (e.g., multigrid).

Recall: Developing better algorithms is as important as better hardware!!!

Introduction to OpenMP

OpenMP

OpenMP: Open specification for Multi-Processing

Use to create code that can run in parallel on a shared memory machine (e.g., multi-core architectures).

OpenMP is an API. Application Programming Interface (API) is a set of routines, protocols, and tools for building software applications.

Can be used with Fortran, C, and C++.

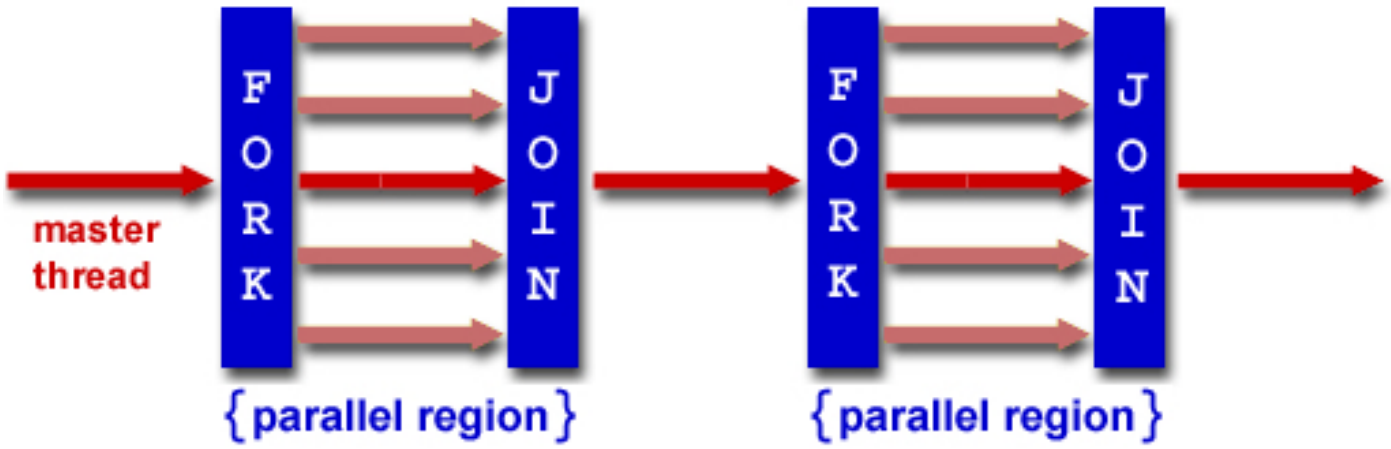
Some references:

- `http://www.openmp.org`
- `http://www.openmp.org/wp/resources`
- B. Chapman, G. Jost, R. van der Pas, *Using OpenMP: Portable Shared Memory Parallel Programming*, 2007.
- R. Chandra, L. Dagum, et. al., *Parallel Programming with OpenMP*, Academic Press, 2001.

OpenMP: Basic idea

Explicit programmer control of parallelization using fork-join model of parallel execution

- all OpenMP programs begin as single process, the master thread, which executes until a parallel region construct is encountered
- **Fork:** master thread creates a team of parallel thread.
- **Join:** when threads complete statements in parallel region construct they synchronize and terminate, leaving only the master thread.



OpenMP: Basic idea

- Rule of thumb: one thread per processor (or core)
- User inserts compiler directives telling compiler how statemnts are to be executed
  - which parts are parallel
  - how to code in parallel regions to threads
  - what data is private (local) to threads
- Compiler generates explicit threaded code
- Dependencies in parallel parts require synchronization between threads
- User's job to remove dependencies in parallel parts or use synchronization.

Simple example #1

$ISUHPC/lectures/lecture17/codes/hello1/omp_hello.c:

```
#include <stdio.h>
#include <stdlib.h>
#ifdef _OPENMP
#  include <omp.h>
#endif

//#include <omp.h>

int main()
{
   void hello();

   printf("\n");

#  pragma omp parallel
   hello();

   printf("\n");
```

Simple example #1

$ISUHPC/lectures/lecture17/codes/hello1/omp_hello.c:

```
}

void hello()
{
#   ifdef _OPENMP
        const int my_rank = omp_get_thread_num();
        const int actual_thread_count = omp_get_num_threads();
#   else
        const int my_rank = 0;
        const int actual_thread_count = 1;
#   endif

    printf(" Hello from thread %i of %i\n", my_rank+1,
        actual_thread_count);
}
```

Simple example #1–Clang gcc

$ISUHPC/lectures/lecture17/codes/hello1/Makefile:

```
CC = gcc
FFLAGS = -Wall -std=c99 -Xpreprocessor  -fopenmp
LFLAGS = -lomp


OBJECTS = omp_hello.o

main.exe:  $(OBJECTS)
        $(CC) $(LFLAGS) $(OBJECTS) -o main.exe

%.o: %.c
        $(CC) $(FFLAGS) -c  $<

clean:
        rm -f $(OBJECTS) *.exe
```

Simple example #1–GNU gcc

$ISUHPC/lectures/lecture17/codes/hello1/Makefile:

```
CC = gcc

FFLAGS = -Wall -std=c99 -fopenmp
LFLAGS = -lgomp

OBJECTS = omp_hello.o

main.exe:  $(OBJECTS)
        $(CC) $(LFLAGS) $(OBJECTS) -o main.exe

%.o: %.c
        $(CC) $(FFLAGS) -c  $<

clean:
        rm -f $(OBJECTS) *.exe
```

Simple example #1

```
$ make
gcc -Wall -std=c99 -fopenmp -c omp_hello.c
gcc -lgomp omp_hello.o -o main.exe

$ main.exe

 Hello from thread 2 of 8
 Hello from thread 5 of 8
 Hello from thread 7 of 8
 Hello from thread 1 of 8
 Hello from thread 8 of 8
 Hello from thread 3 of 8
 Hello from thread 4 of 8
 Hello from thread 6 of 8
```

Simple example #2

$ISUHPC/lectures/lecture17/codes/hello2/omp_hello.c:

```
#include <stdio.h>
#include <stdlib.h>
#ifdef _OPENMP
#  include <omp.h>
#endif

int main(int argc, char* argv[])
{
   void usage(const char* prog_name);
   void hello(const int thread_count);

   if (argc != 2) { usage(argv[0]); }
   const int thread_count = strtol(argv[1], NULL, 10);
   if (thread_count <= 0)
   { usage(argv[0]); }

   printf("\n");
#  pragma omp parallel num_threads(thread_count)
   hello(thread_count);
   printf("\n");

   return 0;
}
```

Simple example #2

$ISUHPC/lectures/lecture17/codes/hello2/omp_hello.c:

```
void usage(const char *prog_name)
{
   fprintf(stderr, "usage: %s <thread_count>\n", prog_name);
   fprintf(stderr, "   thread_count should be positive\n");
   exit(1);
}
void hello(const int thread_count)
{
#  ifdef _OPENMP
      const int my_rank = omp_get_thread_num();
      const int actual_thread_count = omp_get_num_threads();
#  else
      const int my_rank = 0;
      const int actual_thread_count = 1;
#  endif
   if (my_rank == 0 && thread_count != actual_thread_count)
   {
      fprintf(stderr, "Number of threads started != %i\n",
              thread_count);
   }
   printf(" Hello from thread %i of %i\n", my_rank+1,
          actual_thread_count);
}
```

Simple example #2–Clang gcc

$ISUHPC/lectures/lecture17/codes/hello2/Makefile:

```
CC = gcc
FFLAGS = -Wall -std=c99 -Xpreprocessor -fopenmp
LFLAGS = -lomp
OBJECTS = omp_hello.o

main.exe:  $(OBJECTS)
        $(CC) $(LFLAGS) $(OBJECTS) -o main.exe

%.o: %.c
        $(CC) $(FFLAGS) -c $<

clean:
        rm -f $(OBJECTS) *.exe
```

Simple example #2-GNU gcc

$ISUHPC/lectures/lecture17/codes/hello2/Makefile:

```
CC = gcc
FFLAGS = -Wall -std=c99 -fopenmp
LFLAGS = -lgomp
OBJECTS = omp_hello.o

main.exe:  $(OBJECTS)
        $(CC) $(LFLAGS) $(OBJECTS) -o main.exe

%.o: %.c
        $(CC) $(FFLAGS) -c $<

clean:
        rm -f $(OBJECTS) *.exe
```

Simple example #2

```
1 int main(int argc, char* argv[])
```

```
$ main.exe 5
argc
    2
argv[0]
    main.exe
argv[1]
    5
```

Simple example #2

```c
int main(int argc, char* argv[])
```

```
$ main.exe 5
argc
    2
argv[0]
    main.exe
argv[1]
    5
```

```
$ make
gcc -Wall -std=c99 -fopenmp -c omp_hello.c
gcc -lgomp omp_hello.o -o main.exe

$ main.exe 5

 Hello from thread 2 of 5
 Hello from thread 3 of 5
 Hello from thread 4 of 5
 Hello from thread 1 of 5
 Hello from thread 5 of 5
```

Simple example #3

$ISUHPC/lectures/lecture17/codes/forloop/forloop.c:

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include "matrix.h"
#include <math.h>
#ifdef _OPENMP
#  include <omp.h>
#endif

int main(int argc, char* argv[])
{
   void usage(const char* prog_name);

   if (argc != 3) { usage(argv[0]); }
   const int thread_count = strtol(argv[1], NULL, 10);
   const int N = strtol(argv[2], NULL, 10);
   if (thread_count <= 0)
   { usage(argv[0]); }

   srand( time(NULL) );
   vector v = new_vector(N);
```

Simple example #3

$ISUHPC/lectures/lecture17/codes/forloop/forloop.c:

```
    const double time1 = omp_get_wtime();

#   pragma omp parallel for num_threads(thread_count)
    for (int i=1; i<=N; i++)
    {
        const double x = ((double)(i))/((double)(N));
        double sum = 1.0;
        for (int k=1; k<=200; k++)
        {
            sum += sqrt(pow(x,k+2) + pow(x,k+1) + pow(x,k) + pow(
                x,k-1));
        }
        vget(v,i) = sum;
    }

    const double time2 = omp_get_wtime();
    const double clock_time = time2-time1;

    printf(" With %i threads, clock_time = %11.5e (sec)\n",
                thread_count,clock_time);
```

Simple example #3

$ISUHPC/lectures/lecture17/codes/forloop/forloop.c:

```
    delete_vector(&v);
    return 0;
}
void usage(const char *prog_name)
{
    fprintf(stderr, "usage: %s <thread_count> <matrix size>\n",
         prog_name);
    fprintf(stderr, "   thread_count should be positive\n");
    fprintf(stderr, "   N should be positive\n");
    exit(1);
}
```

Simple example #3

```
$ make
gcc -Wall -O3 -std=c99 -fopenmp -c forloop.c
gcc -lgomp forloop.o matrix.o -o main.exe

$ main.exe 1 10000
 With 1 threads, clock_time = 2.90895e-01 (sec)
$ main.exe 2 10000
 With 2 threads, clock_time = 1.48743e-01 (sec)
$ main.exe 3 10000
 With 3 threads, clock_time = 1.03823e-01 (sec)
$ main.exe 4 10000
 With 4 threads, clock_time = 8.04961e-02 (sec)
$ main.exe 5 10000
 With 5 threads, clock_time = 7.54271e-02 (sec)
$ main.exe 6 10000
 With 6 threads, clock_time = 7.25641e-02 (sec)
$ main.exe 7 10000
 With 7 threads, clock_time = 6.76389e-02 (sec)
$ main.exe 8 10000
 With 8 threads, clock_time = 6.46479e-02 (sec)
```

Using the ISU HPC-Class Cluster

HPC-Class: Education Cluster

IOWA STATE UNIVERSITY
OF SCIENCE AND TECHNOLOGY

HPC-Class (`http://www.hpc.iastate.edu`)

HPC-Class is part of Nova Cluster now: `https://www.hpc.iastate.edu/guides/nova`. Also see the Announcement (Canvas)

HPC-Class: Education Cluster

- SSH to HPC-Class cluster (will need VPN if you are off-campus)

```
ssh {netid}@nova.its.iastate.edu
```

- Use your ISU ID and password
- Type `bash` to get into a bash shell
- Files can be transferred by `scp`:

```
scp <FILENAME> nova.its.iastate.edu:{path to your directory}
```

- Compile as usual
- Execution requires the use of `sbatch` command and a SBATCH script:

https://www.hpc.iastate.edu/guides/nova/slurm-script-generator-for-nova

Help can be found here

https://www.hpc.iastate.edu/about/#community

Using the `sbatch` command

Let's move to the HPC script generator

https://www.hpc.iastate.edu/guides/nova/slurm-script-generator-for-nova

Create an ascii file (let's call it myjob)

Execute using `sbatch` command

```
$ make -f MakefileHPC
gcc -Wall -std=c99 -fopenmp -c omp_hello.c
gcc -lgomp omp_hello.o -o main.exe

$ sbatch myjob

$ squeue -u (your netid)
```

Execute using `sbatch` command

```
$ more output.txt

Hello from thread 9 of 16
Hello from thread 14 of 16
Hello from thread 11 of 16
Hello from thread 8 of 16
Hello from thread 13 of 16
Hello from thread 15 of 16
Hello from thread 2 of 16
Hello from thread 7 of 16
Hello from thread 3 of 16
Hello from thread 12 of 16
Hello from thread 16 of 16
Hello from thread 4 of 16
Hello from thread 6 of 16
Hello from thread 1 of 16
Hello from thread 5 of 16
Hello from thread 10 of 16
```

Lab Assignment: Image Blurring

- Generate a random image of size $N \times N$ with
  `img[i][j] = rand() % 256;`
  Apply a 3×3 filter to obtain the blurred image of size $(N-2) \times (N-2)$. For the filter, for example, at each pixel, using average of the neighbors and itself.
- Implement the problem with number of threads = $1, 2, 4$ and 8, and measure walltime.
- Submit source codes and screenshots of the results. Update Git.